

数字部件设计 Project

单周期 CPU 电路设计

一、项目介绍、要求和提示

在这个项目中，我们最终需要完成一个能够实现指定 RISC-V 指令功能的简单单周期 CPU 的电路设计。附录中，提供了关于这两者的介绍和一个单周期 CPU 电路设计的实例。当然，我们需要完成的工作比这个实例还要简单得多，因为我们只要求完成的单周期 CPU 接收并正确处理 RISC-V 指令的一个很小的子集，同时也减少了对电路中存储设备规模的要求。

对于这个 32 位单周期 CPU 电路，我们有一些具体的要求：

1. 该 CPU 需要完成的指令有以下 9 个：

add, sub, or, slt, addi, ori, slti, sw, lw

每条指令长度都为 32bits，具体细节参考附件或者说明文档。

2. 该 CPU 的部件资源包括：1) X0-X31 共 32 通用寄存器；2) 特殊寄存器 PC (program counter) 和指令暂存寄存器 IR (instruction Register) (注意，IR 不是必须，也可以直接使用 PM memory 存放的指令进行译码)。以上寄存器都是 32bits 字长。

3. 存储器包括 256bytes 的 PM memory 和 256bytes 的 DM memory (地址都为 0~255，采用 little endian 方式存储数据或者指令)。

4. 最后的波形仿真应当采用功能仿真，且所有存储器件中的数据都应当被显示。

设计的正确性，将由功能仿真的结果来检验。一方面，大家应当自行进行功能仿真的测试，并将结果提交；另一方面，我们也会对大家提交的程序进行相关测试。

二、项目提交

1. 最终提交时间为**期末考试之前**，提交内容包括：所有基本部件的程序代码（以 Project 的形式，不能只有.v 文件），每条指令的执行仿真波形图（加简要说明），实验报告和文件清单，所有内容应当被打成一个压缩包上传，文件名为“学号+姓名+prj”。

注意事项：

- CPU 设计可以直接使用表达式实现，但是推荐使用经典的多部件组合方式，具体参考附件 I。如果使用多个基本部件实现，要对各个部件功能进行说明。
- 实验报告包括但不限于对功能仿真波形图的介绍和说明，以及指令处理的具体过程，以及对 CPU 所能完成的所有功能和相关性能分析。
- 可以分两阶段实现：第一步指令存放在 IR 寄存器，直接对单条命令解析实现所需功能；第二步，若干条指令代码存放在存储器从 0x00 开始的存储空间，根据 PC 寄存器的值进行指令读取到 IR，再解析执行，同时更新 PC 寄存器指向下一条指令。
- 实现上面的 9 条指令后，如果有余力，可以尝试实现 beq 指令（有额外加分）。
- 在设计文档的内容中包括但不限于 CPU 的电路示意图、CPU 如何实现其功能、怎样通过改进设计提高其性能以及自己在项目中的体会和感受。

三、项目的评分

项目的基准分为 100 分，主要包括对以下几项的考核：

- 1) 指令功能的实现（程序代码）每条 5 分，共 45 分；
- 2) 指令实现设计思路与仿真结果（波形图或者输出结果），共 30 分；
- 3) 实验报告及感想体会，共 25 分；
- 4) 额外实现 beq 指令（主要是 pc 寄存器更新策略），附加分 10 分。

程序会考察程序的正确、规范等性质；波形图会考察其代表性和体现出的程序性能；文档会考察文档内容是否完整，是否能体现出正确的设计思路，以及是否对单周期 CPU 的实现进行了自己的思考等。本 project 对文档有较高的要求，如果有同学能够提出更好的改进意见和建议，请在文档中特别说明，会酌情予以加分。

I. RISC-V 指令集简介

RISC-V 是很流行的一种 RISC (Reduced Instruction Set Computing) 处理器，指令集具有以下特点：

- 1. 简单的 LOAD / STORE 结构。所有的计算类型的指令均只涉及寄存器堆的存取，只有 LOAD 和 STORE 指令涉及存储器访问。
- 2. 易于流水线 CPU 的设计。RV32I 指令集中所有的指令均为 32 位，而且指令操作码也处于固定位置。
- 3. RISC-V 指令的寻址方式和指令操作上的简易性适合编译器的开发。

RISC-V 有 X0~X31 共 32 个通用寄存器，统称寄存器堆 (register file)，其中寄存器 X0 的内容总是 0。

RV32I 的指令格式有 6 种：

R (register) 型：寄存器操作数指令；**I (immediate) 型**：短立即数操作或者取数指令；**S (store) 型**：存数指令；**B (branch) 型**：条件跳转指令；**J 型**：无条件跳转；**U 型**：长立即数操作指令。指令格式如下：

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0		
funct7				rs2		rs1		funct3		rd			opcode		R-type	
imm[11:0]						rs1		funct3		rd			opcode		I-type	
imm[11:5]				rs2		rs1		funct3		imm[4:0]			opcode		S-type	
imm[12]		imm[10:5]			rs2		rs1		funct3		imm[4:1]		imm[11]		opcode	B-type
imm[31:12]										rd			opcode		U-type	
imm[20]		imm[10:1]			imm[11]		imm[19:12]			rd			opcode		J-type	

其中，opcode 是指令操作码；rs (register source) 是源操作数的寄存器号；rd (register destination) 是目的寄存器号；funct (function) 是扩展的操作码；imm 为立即数，[]里的值为相应的 bit 位。更多的关于 RISC-V 指令集的内容，可以通过查阅参考文献了解。实验需要实现的指令具体规范如下：

add rd, rs1, rs2 x[rd] = x[rs1] + x[rs2]

加 (Add). R-type, RV32I and RV64I.
把寄存器 x[rs2] 加到寄存器 x[rs1] 上，结果写入 x[rd]。忽略算术溢出。
压缩形式：c.add rd, rs2; c.mv rd, rs2

31	25	24	20	19	15	14	12	11	7	6	0
0000000			rs2		rs1		000		Rd		0110011

addi rd, rs1, immediate x[rd] = x[rs1] + sext(immediate)

加立即数 (Add Immediate). I-type, RV32I and RV64I.
把符号位扩展的立即数加到寄存器 x[rs1] 上，结果写入 x[rd]。忽略算术溢出。
压缩形式：c.li rd, imm; c.addi rd, imm; c.addi16sp imm; c.addi4spn rd, imm

31	20	19	15	14	12	11	7	6	0		
immediate[11:0]				rs1		000		rd		0010011	

sub rd, rs1, rs2

$$x[rd] = x[rs1] - x[rs2]$$

减(*Subtract*). R-type, RV32I and RV64I.

$x[rs1]$ 减去 $x[rs2]$, 结果写入 $x[rd]$ 。忽略算术溢出。

压缩形式: **c.sub** rd, rs2

31	25 24	20 19	15 14	12 11	7 6	0
0100000	rs2	rs1	000	rd	0110011	

or rd, rs1, rs2

$$x[rd] = x[rs1] | x[rs2]$$

取或(*OR*). R-type, RV32I and RV64I.

把寄存器 $x[rs1]$ 和寄存器 $x[rs2]$ 按位取或, 结果写入 $x[rd]$ 。

压缩形式: **c.or** rd, rs2

31	25 24	20 19	15 14	12 11	7 6	0
0000000	rs2	rs1	110	rd	0110011	

ori rd, rs1, immediate

$$x[rd] = x[rs1] | \text{sext}(\text{immediate})$$

立即数取或(*OR Immediate*). R-type, RV32I and RV64I.

把寄存器 $x[rs1]$ 和有符号扩展的立即数 *immediate* 按位取或, 结果写入 $x[rd]$ 。

压缩形式: **c.or** rd, rs2

31	25 24	20 19	15 14	12 11	7 6	0
Immediate[11:0]	rs2	rs1	110	rd	0010011	

注: 上面 ori 图示有误, bit20-31 应该为 Immediate[11:0]

slt rd, rs1, rs2

$$x[rd] = (x[rs1] <_s x[rs2])$$

小于则置位(*Set if Less Than*). R-type, RV32I and RV64I.

比较 $x[rs1]$ 和 $x[rs2]$ 中的数, 如果 $x[rs1]$ 更小, 向 $x[rd]$ 写入 1, 否则写入 0。

31	25 24	20 19	15 14	12 11	7 6	0
0000000	rs2	rs1	010	rd	0110011	

slti rd, rs1, immediate

$$x[rd] = (x[rs1] <_s \text{sext}(\text{immediate}))$$

小于立即数则置位(*Set if Less Than Immediate*). I-type, RV32I and RV64I.

比较 $x[rs1]$ 和有符号扩展的 *immediate*, 如果 $x[rs1]$ 更小, 向 $x[rd]$ 写入 1, 否则写入 0。

31	20 19	15 14	12 11	7 6	0
immediate[11:0]	rs1	010	rd	0010011	

SW $rs2, offset(rs1)$ $M[x[rs1] + sext(offset) = x[rs2][31:0]$

存字 (*Store Word*). S-type, RV32I and RV64I.

将 $x[rs2]$ 的低位 4 个字节存入内存地址 $x[rs1] + sign-extend(offset)$ 。

压缩形式: **c.swsp** $rs2, offset$; **c.sw** $rs2, offset(rs1)$

31	25 24	20 19	15 14	12 11	7 6	0
offset[11:5]	rs2	rs1	010	offset[4:0]	0100011	

lw $rd, offset(rs1)$ $x[rd] = sext(M[x[rs1] + sext(offset))][31:0]$

字加载 (*Load Word*). I-type, RV32I and RV64I.

从地址 $x[rs1] + sign-extend(offset)$ 读取四个字节，写入 $x[rd]$ 。对于 RV64I，结果要进行符号位扩展。

压缩形式: **c.lwsp** $rd, offset$; **c.lw** $rd, offset(rs1)$

31	20 19	15 14	12 11	7 6	0
offset[11:0]	rs1	010	rd	0000011	

beq $rs1, rs2, offset$ $if (rs1 == rs2) pc += sext(offset)$

相等时分支 (*Branch if Equal*). B-type, RV32I and RV64I.

若寄存器 $x[rs1]$ 和寄存器 $x[rs2]$ 的值相等，把 pc 的值设为当前值加上符号位扩展的偏移 $offset$ 。

压缩形式: **c.beqz** $rs1, offset$

31	25 24	20 19	15 14	12 11	7 6	0
offset[12 10:5]	rs2	rs1	000	offset[4:1 11]	1100011	

II.单周期 CPU 简介

单周期 CPU 的特点是每条指令的执行需要一个时钟周期，只有一条指令执行完后才能执行下一条指令，由于每个时钟周期的时间长短是相同的，因此，在确定时钟周期的时间长度时，要以指令集中最复杂的指令需要执行的时间为第一考虑。

单周期 CPU 使用 PC 作为地址从程序存储器中获得指令，根据指令的意义完成相应操作，在时钟上升沿将结果写入寄存器，与此同时，PC 也一并被更新。

下图是一个简单的单周期 CPU 的电路图。主要元件有 PC、程序存储器，寄存器堆，ALU，数据存储器和控制部件等。

注意：下述说明给出的 9 条指令与实验要求不完全一致，仅供参考。

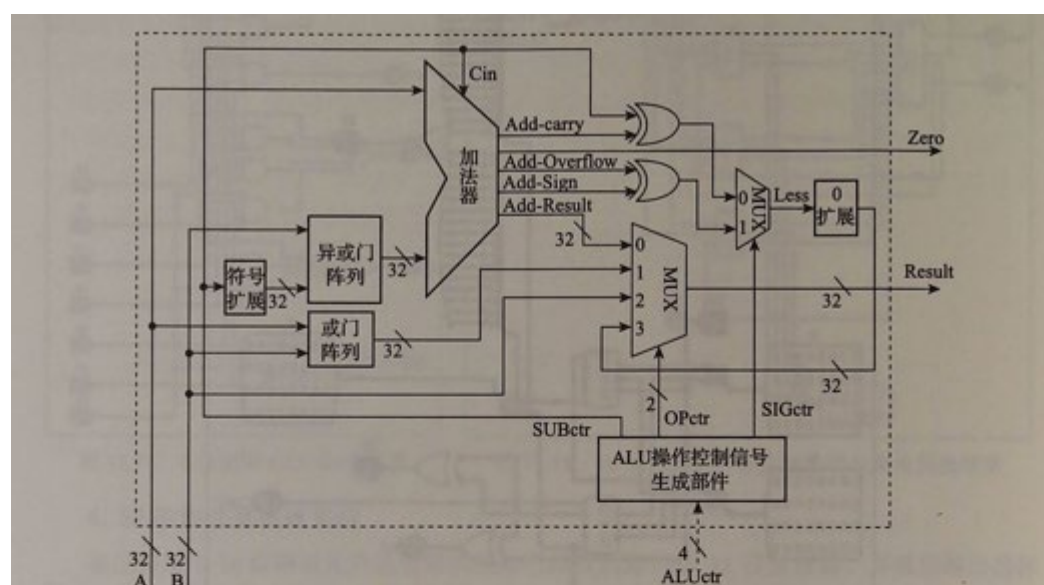
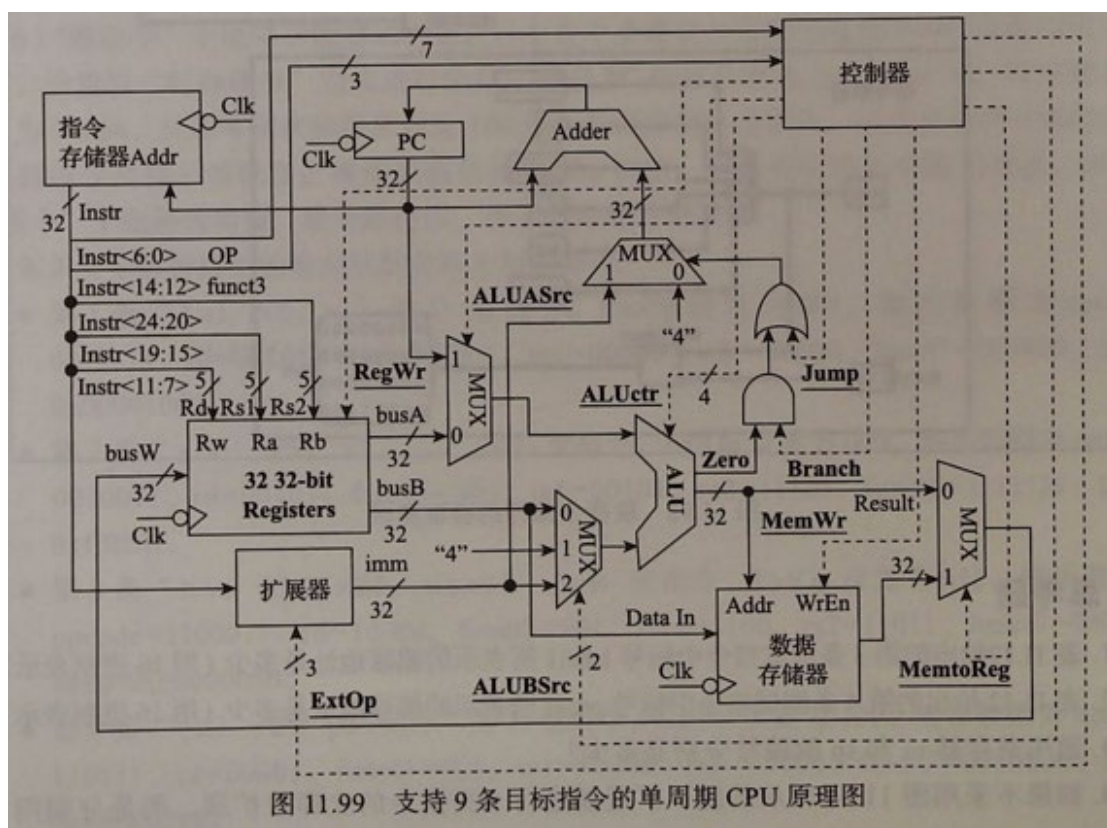


图 11.78 支持 9 条目标指令的 ALU 的原理图

表 11.10 9 条目标指令的功能及 ALU 操作控制信号的取值

指令	功能	操作类型	SUBctr	SIGctr	OPctr<1:0>
add rd, rs1, rs2	$R[rd] \leftarrow R[rs1] + R[rs2]$	加	0	x	00
slt rd, rs1, rs2	if ($R[rs1] < R[rs2]$) $R[rd] \leftarrow 1$ else $R[rd] \leftarrow 0$	减，带符号整数 比较大小	1	1	11
sltu rd, rs1, rs2	if ($R[rs1] < R[rs2]$) $R[rd] \leftarrow 1$ else $R[rd] \leftarrow 0$	减，无符号数比 较大小	1	0	11
ori rt, rs1, imm12	$R[rt] \leftarrow R[rs1] \mid \text{SEXT}(\text{imm12})$	按位或	x	x	01
lui rd, imm20	$R[rt] \leftarrow \text{imm20} \parallel 000H$	选择操作数 B	x	x	10
lw rd, imm12(rs1)	$\text{Addr} \leftarrow R[rs1] + \text{SEXT}(\text{imm12})$ $R[rd] \leftarrow M[\text{Addr}]$	加	0	x	00
sw rs2, imm12(rs1)	$\text{Addr} \leftarrow R[rs1] + \text{SEXT}(\text{imm12})$ $M[\text{Addr}] \leftarrow R[rs2]$	加	0	x	00
beq rs1, rs2, imm12	$\text{Cond} \leftarrow R[rs1] - R[rs2]$ if ($\text{Cond} \text{ eq } 0$) $\text{PC} \leftarrow \text{PC} + (\text{SEXT}(\text{imm12}) \times 2)$	减 (判 0)	1	x	x x
jal rd, imm20	$R[rd] \leftarrow \text{PC} + 4$ $\text{PC} \leftarrow \text{PC} + (\text{SEXT}(\text{imm20}) \times 2)$	加	0	x	00



1. 控制器设计实验

对于图 11.99 中的单周期 CPU，表 11.13 给出了所支持的 9 条目标指令所对应的控制信号取值。

表 11.13 9 条目标指令的控制信号取值

控制信号	func3	000	010	011	110	无关	010	010	000	无关
	op	0110011	0110011	0110011	0010011	0110111	0000011	0100011	1100011	1101111
		add	slt	sltu	ori	lui	lw	sw	beq	jal
Branch		0	0	0	0	0	0	0	1	0
Jump		0	0	0	0	0	0	0	0	1
ALUASrc		0	0	0	0	x	0	0	0	1
ALUBSrc<1:0>		00	00	00	10	10	10	10	00	01
ALUctr<3:0>		0000 (add)	0010 (slt)	0011 (sltu)	0110 (or)	1111 (srcB)	0000 (add)	0000 (add)	1000 (sub)	0000 (add)
MemtoReg		0	0	0	0	0	1	x	x	0
RegWr		1	1	1	1	1	1	0	0	1
MemWr		0	0	0	0	0	0	1	0	0
ExtOp<2:0>		x	x	x	000 immI	001 immU	000 immI	010 immS	011 immB	100 immJ

根据表 11.13 给出的指令和控制信号之间的逻辑关系，可得到每个控制信号的逻辑表达式。假定操作码 op 的信息位分别表示为 op<6>、op<5>、op<4>、op<3>、op<2>、op<1> 和 op<0>，则部分控制信号的逻辑表达式如下：

$$\begin{aligned}
 \text{Branch} &= \text{op}<6> \& \text{op}<5> \& \sim\text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0> && (\text{B-型}) \\
 \text{Jump} &= \text{op}<6> \& \text{op}<5> \& \sim\text{op}<4> \& \text{op}<3> \& \text{op}<2> \& \text{op}<1> \& \text{op}<0> && (\text{J-型}) \\
 \text{MemtoReg} &= \sim\text{op}<6> \& \sim\text{op}<5> \& \sim\text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0> && (\text{Load}) \\
 \text{RegWr} &= (\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) && (\text{R-型}) \\
 &\quad | (\sim\text{op}<6> \& \sim\text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) && (\text{I-型-ALU}) \\
 &\quad | (\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \text{op}<2> \& \text{op}<1> \& \text{op}<0>) && (\text{lui}) \\
 &\quad | (\sim\text{op}<6> \& \sim\text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) && (\text{Load}) \\
 &\quad | (\sim\text{op}<6> \& \sim\text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) && (\text{J-型}) \\
 &\quad | (\text{op}<6> \& \text{op}<5> \& \sim\text{op}<4> \& \text{op}<3> \& \text{op}<2> \& \text{op}<1> \& \text{op}<0>) && (\text{lui}) \\
 \text{ALUctr}<3> &= (\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \text{op}<2> \& \text{op}<1> \& \text{op}<0>) && (\text{B-型}) \\
 &\quad | (\text{op}<6> \& \text{op}<5> \& \sim\text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) \\
 \text{ALUctr}<2> &= ((\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) && (\text{R-型+I-型-ALU}) \\
 &\quad | (\sim\text{op}<6> \& \sim\text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) && \& \text{fn}<2> \\
 &\quad | (\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \text{op}<2> \& \text{op}<1> \& \text{op}<0>) && \& \text{fn}<2> + (\text{lui}) \\
 \text{ALUctr}<1> &= ((\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) && (\text{R-型+I-型-ALU}) \\
 &\quad | (\sim\text{op}<6> \& \sim\text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) && \& \text{fn}<1> \\
 &\quad | (\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \text{op}<2> \& \text{op}<1> \& \text{op}<0>) && \& \text{fn}<1> + (\text{lui}) \\
 \text{ALUctr}<0> &= ((\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) && (\text{R-型+I-型-ALU}) \\
 &\quad | (\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \sim\text{op}<2> \& \text{op}<1> \& \text{op}<0>) && \& \text{fn}<0> \\
 &\quad | (\sim\text{op}<6> \& \text{op}<5> \& \text{op}<4> \& \sim\text{op}<3> \& \text{op}<2> \& \text{op}<1> \& \text{op}<0>) && \& \text{fn}<0> + (\text{lui})
 \end{aligned}$$

$$\begin{aligned}
 & | (\sim op<6> \& \sim op<5> \& op<4> \& \sim op<3> \& \sim op<2> \& op<1> \& op<0>) \& fn<0> \\
 & | (\sim op<6> \& op<5> \& op<4> \& \sim op<3> \& op<2> \& op<1> \& op<0>) \\
 & \hspace{10em} (R\text{-型}+I\text{-型-ALU})\& fn<0> + (lui)
 \end{aligned}$$

根据上述各控制信号的逻辑表达式，可得到图 11.100 所示的控制器电路。

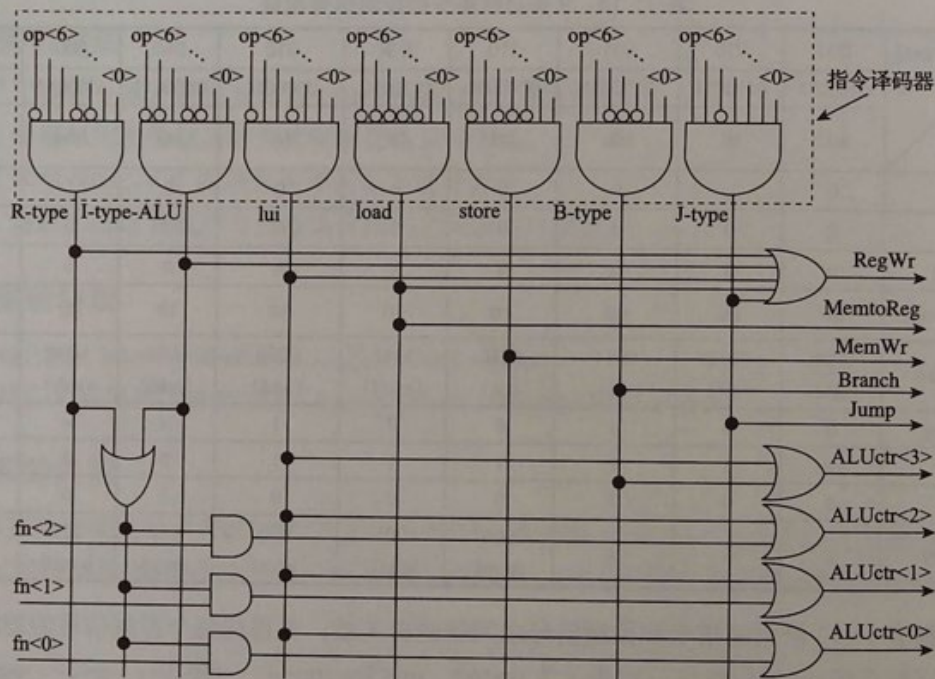


图 11.100 控制器电路原理图

表 11.14 9 条目标指令的格式及其操作码和功能码

指 令	funct7 (31-25)	rs2 (24-20)	rs1 (19-15)	funct3 (14-12)	Rd (11-7)	Op (6-0)
add rd, rs1, rs2	0000 000	rs2	rs1	000	rd	011 0011
slt rd, rs1, rs2	0000 000	rs2	rs1	010	rd	011 0011
sltu rd, rs1, rs2	0000 000	rs2	rs1	011	rd	011 0011
ori rt, rs1, imm12	imm[11:0]		rs1	110	rd	001 0011
lui rd, imm20	imm[31:12]				rd	011 0111
lw rd, rs1, imm12	imm[11:0]		rs1	010	rd	000 0011
sw rs1, rs2, imm12	imm[11:5]	rs2	rs1	010	imm[4:0]	010 0011
beq rs1, rs2, imm12	imm[12][10:5]	rs2	rs1	000	imm[4:1][11]	110 0011
jal rd, imm20	imm[20][10:1][11[19:12]				rd	110 1111

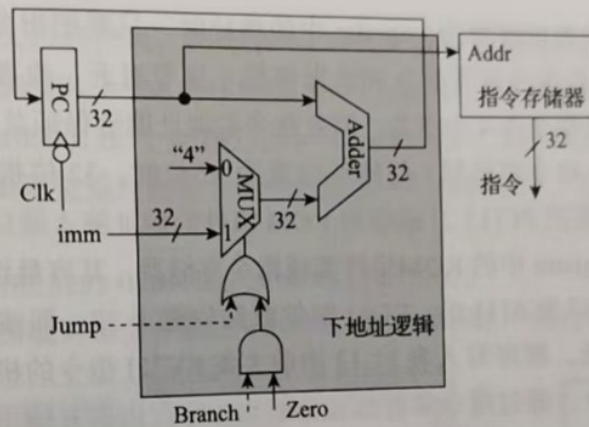


图 11.90 取指令部件原理图

	31	27	26	25	24	20	19	15	14	12	11	7	6	0
R	funct7				rs2		rs1		funct3		rd		opcode	
I	imm[11:0]						rs1		funct3		rd		opcode	
S	imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode	
B	imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode	
U	imm[31:12]										rd		opcode	
J	imm[20 10:1 11 19:12]										rd		opcode	

图 11.91 RISC-V 指令格式

图 11.91 RISC-V 指令格式